uv Guide: New Projects, Commands and Files

Interview revision notes - Python environments & dependencies with uv

1. What is uv?

uv is a fast, all-in-one Python tool (written in Rust, by Astral). It installs Python versions, creates virtual environments, manages dependencies and runs code. It replaces the combination of pyenv + venv + pip + pip-tools + pipx. Recommended approach: **no global Python**; let uv manage the Python version per project.

2. Steps for a new project

```
uv init my-project            # 1. create folder + pyproject.toml (+ starter files)
cd my-project
uv python pin 3.13            # 2. (optional) pin the Python version
uv add requests ipykernel     # 3. add dependencies -> creates .venv and uv.lock
uv add --dev pytest ruff      # 4. add dev-only tools
uv run main.py                # 5. run code inside the project env (no activation)
uv sync                       # 6. rebuild the env from the lockfile (other machine/CI)
```

Existing folder: run `uv init` inside it. Use `uv init --bare --name myproject` to get only pyproject.toml (no main.py / README.md).

3. Creating a venv with a specific Python version

```
uv python install 3.12                 # download that Python version (once)
uv venv --python 3.12                  # create .venv using Python 3.12
uv venv --python 3.12 myenv            # create the venv in a custom folder
uv venv --python 3.13 --no-managed-python   # only use Pythons already on the system
uv venv --python "C:\path\to\python.exe"  # use an exact interpreter

.venv\Scripts\activate                # activate (Windows); optional with uv run
.venv\Scripts\python.exe --version    # verify the version inside the venv
uv pip install requests                # install into the venv (not recorded)
```

- If the requested version is missing, uv downloads it automatically (unless --no-managed-python is used).
- `uv venv` alone creates a plain environment. For a tracked project use `uv init` + `uv python pin` + `uv add`.
- Recreate an existing venv with another version: `uv venv --python 3.13 --clear`.
- A venv is tied to the interpreter it was built from; if that Python is removed, recreate the venv.

4. Significance of project files

File	Purpose	Commit to git?
`pyproject.toml`	Project definition: name, version, required Python, and your **direct** dependencies (what you asked for). Edited by you or by uv add/remove.	Yes
`uv.lock`	Exact resolved versions of **every** package (incl. sub-dependencies) with hashes. Gives reproducible installs. Managed by uv; never hand-edit.	Yes
`.python-version`	Python version the project uses. uv reads it and downloads that version if missing.	Yes
`.venv/`	The actual environment with installed packages. Disposable: delete and rebuild with uv sync.	No (.gitignore)
`main.py`	Starter script created by uv init. Rename or delete.	Yes
`README.md`	Project description, referenced by pyproject.toml.	Yes
`.gitignore`	Files git should skip (includes .venv).	Yes

Key relationship

pyproject.toml says *what you want* (requests). **uv.lock** records *exactly what you got* (requests 2.34.2 + its dependencies). **.venv** is *the result* of installing it. Only the first two (plus .python-version) need to be shared.

5. Everyday command reference

Task	Command
New project	`uv init name`
Add / remove a package	`uv add pkg` / `uv remove pkg`
Add dev-only package	`uv add --dev pkg`
Install from requirements.txt	`uv add -r req.txt` (recorded) or `uv pip install -r req.txt` (not recorded)
Run a script	`uv run script.py`
Run with a temporary package	`uv run --with requests script.py`
Rebuild env from lockfile	`uv sync`
Fail if lock is outdated (CI)	`uv sync --locked`
Update locked versions	`uv lock --upgrade` / `--upgrade-package pkg`
Show dependency tree	`uv tree`
Export requirements.txt	`uv export --no-hashes -o requirements.txt`
Create venv with a Python version	`uv venv --python 3.12`
Install a Python version	`uv python install 3.13`
List installed Pythons	`uv python list --only-installed`
Pin project Python	`uv python pin 3.12`
Install a CLI tool (isolated)	`uv tool install ruff`

6. Common gotchas

- **uv venv** only creates an environment: no pyproject.toml, no lockfile, no record of packages.
- **uv pip install** installs but does not update pyproject.toml. Use **uv add** for anything to keep.
- uv venvs have no pip by default. In notebooks use `%pip install pkg` (installs into the active kernel); plain `!pip install` may hit a different Python.
- Prefer stable Python (3.12 / 3.13); pre-release versions often lack package wheels.
- Never edit .venv by hand; change dependencies via uv add / uv remove.
- Never commit secrets (API keys) in notebooks; load them from environment variables or a .env file in .gitignore.

7. Interview quick answers

- **Why a lockfile?** Reproducible, identical installs across machines and CI; pins transitive dependencies.
- **pyproject.toml vs uv.lock?** Intent (loose constraints) vs exact resolved versions.
- **Why uv over pip + venv?** Much faster, one tool, built-in Python version management and locking.
- **How to reproduce an env?** Clone the repo, run `uv sync`.
- **Do you need a global Python?** No. uv downloads and manages versions per project.